

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 8: The Transactional Microservice (Spring Boot + JPA)

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO4
Session	Practical 8	Date	

Aim

To build a Spring Boot inventory-service (Spring Web, Spring Data JPA, PostgreSQL Driver, Validation) with Inventory and Order entities, JpaRepository repositories, a service that allows a borrow only when availableStock > 0, POST /orders and GET /inventory/{bookId} endpoints, JSR-380 validation with a @ControllerAdvice error handler, and to route requests to it through the FastAPI Gateway.

Program

Project generated with Spring Initializr (Maven, Java 17, Spring Boot 3.3.5). Run with mvn spring-boot:run, then start the gateway with uvicorn gateway:app --port 8000.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>

  <groupId>com.bookflow</groupId>
  <artifactId>inventory-service</artifactId>
  <version>1.0.0</version>
  <name>inventory-service</name>
  <description>BookFlow inventory and order microservice</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

```

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>
  </project>

```

src/main/resources/application.yml

```

server:
  port: 8080

spring:
  application:
    name: inventory-service
  datasource:
    url: jdbc:postgresql://localhost:5432/bookflow_db
    username: postgres
    password: postgres
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update          # creates/updates the inventory and orders tables
      show-sql: false
      open-in-view: false
      properties:
        hibernate:
          format_sql: true

```

InventoryServiceApplication.java

```

package com.bookflow.inventory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class InventoryServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(InventoryServiceApplication.class, args);
    }
}

```

entity/Inventory.java

```

package com.bookflow.inventory.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;

/** Stock level of one book. bookId is the ID of the book in the FastAPI catalog service. */
@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    @Column(name = "book_id")
    private Long bookId;

    @NotNull
    @Min(value = 0, message = "totalStock cannot be negative")
    @Column(name = "total_stock", nullable = false)
    private Integer totalStock;

    @NotNull
    @Min(value = 0, message = "availableStock cannot be negative")
    @Column(name = "available_stock", nullable = false)
    private Integer availableStock;

    protected Inventory() {
        // required by JPA
    }
}

```

```

    public Inventory(Long bookId, Integer totalStock, Integer availableStock) {
        this.bookId = bookId;
        this.totalStock = totalStock;
        this.availableStock = availableStock;
    }

    public Long getBookId() { return bookId; }
    public Integer getTotalStock() { return totalStock; }
    public Integer getAvailableStock() { return availableStock; }

    public void setTotalStock(Integer totalStock) { this.totalStock = totalStock; }
    public void setAvailableStock(Integer availableStock) { this.availableStock = availableStock; }
}

```

entity/BookOrder.java

```

package com.bookflow.inventory.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.LocalDateTime;

/**
 * One borrow order. userId comes from the Node.js user-service (MongoDB ObjectId),
 * bookId comes from the FastAPI catalog service. The table is named "orders"
 * because ORDER is a reserved word in SQL.
 */
@Entity
@Table(name = "orders")
public class BookOrder {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", nullable = false, length = 24)
    private String userId;

    @Column(name = "book_id", nullable = false)
    private Long bookId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private OrderStatus status;

    @Column(name = "borrowed_at", nullable = false)
    private LocalDateTime borrowedAt;

    @Column(name = "returned_at")
    private LocalDateTime returnedAt;

    protected BookOrder() {
        // required by JPA
    }

    public BookOrder(String userId, Long bookId) {
        this.userId = userId;
        this.bookId = bookId;
        this.status = OrderStatus.BORROWED;
        this.borrowedAt = LocalDateTime.now();
    }

    public void markReturned() {
        this.status = OrderStatus.RETURNED;
        this.returnedAt = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public String getUserId() { return userId; }
    public Long getBookId() { return bookId; }
    public OrderStatus getStatus() { return status; }
    public LocalDateTime getBorrowedAt() { return borrowedAt; }
}

```

```
    public LocalDateTime getReturnedAt() { return returnedAt; }
}
```

entity/OrderStatus.java

```
package com.bookflow.inventory.entity;

public enum OrderStatus {
    BORROWED,
    RETURNED
}
```

repository/InventoryRepository.java

```
package com.bookflow.inventory.repository;

import com.bookflow.inventory.entity.Inventory;
import jakarta.persistence.LockModeType;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface InventoryRepository extends JpaRepository<Inventory, Long> {

    /** SELECT ... FOR UPDATE: prevents two users borrowing the last copy at the same time. */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT i FROM Inventory i WHERE i.bookId = :bookId")
    Optional<Inventory> findByIdForUpdate(@Param("bookId") Long bookId);
}
```

repository/OrderRepository.java

```
package com.bookflow.inventory.repository;

import com.bookflow.inventory.entity.BookOrder;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<BookOrder, Long> {

    List<BookOrder> findByUserIdOrderByBorrowedAtDesc(String userId);
}
```

dto/OrderRequest.java

```
package com.bookflow.inventory.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

public record OrderRequest(
    @NotBlank(message = "userId is required") String userId,
    @NotNull(message = "bookId is required") @Positive(message = "bookId must be positive") Long bookId) {
}
```

dto/InventoryRequest.java

```
package com.bookflow.inventory.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

public record InventoryRequest(
    @NotNull(message = "bookId is required") @Positive(message = "bookId must be positive") Long bookId,
    @NotNull(message = "totalStock is required") @Min(value = 0, message = "totalStock cannot be negative")
    Integer totalStock) {
}
```

service/LibraryService.java

```
package com.bookflow.inventory.service;

import com.bookflow.inventory.dto.InventoryRequest;
import com.bookflow.inventory.dto.OrderRequest;
import com.bookflow.inventory.entity.BookOrder;
import com.bookflow.inventory.entity.Inventory;
import com.bookflow.inventory.entity.OrderStatus;
import com.bookflow.inventory.exception.InvalidOperationException;
import com.bookflow.inventory.exception.OutOfStockException;
```

```

import com.bookflow.inventory.exception.ResourceNotFoundException;
import com.bookflow.inventory.repository.InventoryRepository;
import com.bookflow.inventory.repository.OrderRepository;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/** Business logic for stock management and borrowing. */
@Service
public class LibraryService {

    private final InventoryRepository inventoryRepository;
    private final OrderRepository orderRepository;

    // Constructor injection: Spring's IoC container supplies both repositories
    public LibraryService(InventoryRepository inventoryRepository, OrderRepository orderRepository) {
        this.inventoryRepository = inventoryRepository;
        this.orderRepository = orderRepository;
    }

    @Transactional(readOnly = true)
    public Inventory getInventory(Long bookId) {
        return inventoryRepository.findById(bookId)
            .orElseThrow(() -> new ResourceNotFoundException("No inventory record for book " + bookId));
    }

    @Transactional
    public Inventory addOrUpdateStock(InventoryRequest request) {
        Inventory inventory = inventoryRepository.findById(request.bookId())
            .map(existing -> {
                int borrowed = existing.getTotalStock() - existing.getAvailableStock();
                if (request.totalStock() < borrowed) {
                    throw new InvalidOperationException(
                        "totalStock cannot be less than the " + borrowed + " copies currently"
                        + borrowed);
                }
                existing.setTotalStock(request.totalStock());
                existing.setAvailableStock(request.totalStock() - borrowed);
                return existing;
            })
            .orElseGet(() -> new Inventory(request.bookId(), request.totalStock(), request.totalStock()));
        return inventoryRepository.save(inventory);
    }

    /** Borrow a book only if availableStock > 0. Both writes commit or roll back together. */
    @Transactional
    public BookOrder borrowBook(OrderRequest request) {
        Inventory inventory = inventoryRepository.findByIdForUpdate(request.bookId())
            .orElseThrow(() -> new ResourceNotFoundException("No inventory record for book " +
                request.bookId()));

        if (inventory.getAvailableStock() <= 0) {
            throw new OutOfStockException(request.bookId());
        }
        inventory.setAvailableStock(inventory.getAvailableStock() - 1);
        return orderRepository.save(new BookOrder(request.userId(), request.bookId()));
    }

    @Transactional
    public BookOrder returnBook(Long orderId) {
        BookOrder order = orderRepository.findById(orderId)
            .orElseThrow(() -> new ResourceNotFoundException("Order " + orderId + " not found"));
        if (order.getStatus() == OrderStatus.RETURNED) {
            throw new InvalidOperationException("Order " + orderId + " has already been returned");
        }
        Inventory inventory = inventoryRepository.findByIdForUpdate(order.getBookId())
            .orElseThrow(() -> new ResourceNotFoundException("No inventory record for book " +
                order.getBookId()));
        inventory.setAvailableStock(inventory.getAvailableStock() + 1);
        order.markReturned();
        return order;
    }

    @Transactional(readOnly = true)
    public List<BookOrder> ordersForUser(String userId) {
        return orderRepository.findByUserIdOrderByBorrowedAtDesc(userId);
    }
}

```

controller/InventoryController.java

```
package com.bookflow.inventory.controller;

import com.bookflow.inventory.dto.InventoryRequest;
import com.bookflow.inventory.entity.Inventory;
import com.bookflow.inventory.service.LibraryService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/inventory")
public class InventoryController {

    private final LibraryService libraryService;

    public InventoryController(LibraryService libraryService) {
        this.libraryService = libraryService;
    }

    @GetMapping("/{bookId}")
    public Inventory getInventory(@PathVariable Long bookId) {
        return libraryService.getInventory(bookId);
    }

    @PostMapping
    public ResponseEntity<Inventory> addStock(@Valid @RequestBody InventoryRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(libraryService.addOrUpdateStock(request));
    }
}
```

controller/OrderController.java

```
package com.bookflow.inventory.controller;

import com.bookflow.inventory.dto.OrderRequest;
import com.bookflow.inventory.entity.BookOrder;
import com.bookflow.inventory.service.LibraryService;
import jakarta.validation.Valid;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final LibraryService libraryService;

    public OrderController(LibraryService libraryService) {
        this.libraryService = libraryService;
    }

    @PostMapping
    public ResponseEntity<BookOrder> borrow(@Valid @RequestBody OrderRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(libraryService.borrowBook(request));
    }

    @PutMapping("/{orderId}/return")
    public BookOrder returnBook(@PathVariable Long orderId) {
        return libraryService.returnBook(orderId);
    }

    @GetMapping
```

```

    public List<BookOrder> ordersForUser(@RequestParam String userId) {
        return libraryService.ordersForUser(userId);
    }
}

```

exception/OutOfStockException.java

```

package com.bookflow.inventory.exception;

public class OutOfStockException extends RuntimeException {

    public OutOfStockException(Long bookId) {
        super("Book " + bookId + " is out of stock");
    }
}

```

exception/ResourceNotFoundException.java

```

package com.bookflow.inventory.exception;

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

exception/InvalidOperationException.java

```

package com.bookflow.inventory.exception;

public class InvalidOperationException extends RuntimeException {

    public InvalidOperationException(String message) {
        super(message);
    }
}

```

exception/GlobalExceptionHandler.java

```

package com.bookflow.inventory.exception;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

/** Converts exceptions into clean JSON error responses for every controller. */
@ControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(OutOfStockException.class)
    public ResponseEntity<Map<String, Object>> handleOutOfStock(OutOfStockException ex) {
        return build(HttpStatus.CONFLICT, ex.getMessage(), null);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), null);
    }

    @ExceptionHandler(InvalidOperationException.class)
    public ResponseEntity<Map<String, Object>> handleInvalid(InvalidOperationException ex) {
        return build(HttpStatus.BAD_REQUEST, ex.getMessage(), null);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex) {
        List<String> details = ex.getBindingResult().getFieldErrors().stream()
            .map(error -> error.getField() + ": " + error.getDefaultMessage())
            .sorted()
            .toList();
    }
}

```

```

        return build(HttpStatus.BAD_REQUEST, "Validation failed", details);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleUnexpected(Exception ex) {
        log.error("Unexpected error", ex);
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "Internal server error", null);
    }

    private ResponseEntity<Map<String, Object>> build(HttpStatus status, String message, List<String> details) {
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now().withNano(0).toString());
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", message);
        if (details != null) {
            body.put("details", details);
        }
        return ResponseEntity.status(status).body(body);
    }
}

```

gateway.py (FastAPI Gateway update)

```

"""
FastAPI Gateway - routes inventory and order requests to the Spring Boot
inventory-service (port 8080).

Run: uvicorn gateway:app --port 8000
"""
import os
from typing import Any, Optional

import httpx
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse, Response

INVENTORY_SERVICE_URL = os.getenv("INVENTORY_SERVICE_URL", "http://localhost:8080")
TIMEOUT = httpx.Timeout(5.0, connect=2.0)

app = FastAPI(title="BookFlow Gateway", version="1.1.0")

async def forward(method: str, url: str, json_body: Optional[Any] = None) -> Response:
    """Forward one request to a downstream service and relay its status and body."""
    try:
        async with httpx.AsyncClient(timeout=TIMEOUT) as client:
            upstream = await client.request(method, url, json=json_body)
    except (httpx.ConnectError, httpx.TimeoutException):
        return JSONResponse(
            status_code=503,
            content={"error": "Service Unavailable",
                    "message": "Inventory service is not reachable. Please try again later."},
        )
    return Response(
        content=upstream.content,
        status_code=upstream.status_code,
        media_type=upstream.headers.get("content-type", "application/json"),
    )

@app.get("/api/v1/inventory/{book_id}")
async def get_inventory(book_id: int) -> Response:
    return await forward("GET", f"{INVENTORY_SERVICE_URL}/inventory/{book_id}")

@app.post("/api/v1/inventory")
async def add_stock(request: Request) -> Response:
    return await forward("POST", f"{INVENTORY_SERVICE_URL}/inventory", await request.json())

@app.post("/api/v1/orders")
async def borrow_book(request: Request) -> Response:
    return await forward("POST", f"{INVENTORY_SERVICE_URL}/orders", await request.json())

@app.put("/api/v1/orders/{order_id}/return")
async def return_book(order_id: int) -> Response:
    return await forward("PUT", f"{INVENTORY_SERVICE_URL}/orders/{order_id}/return")

```

Output

Timestamps and generated IDs depend on the run.

Final test: client -> Gateway (8000) -> Spring Boot (8080) -> PostgreSQL

```
$ cd inventory-service && mvn spring-boot:run
...
INFO --- [main] c.b.inventory.InventoryServiceApplication : Starting InventoryServiceApplication using Java 17
INFO --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
INFO --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
INFO --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context
    path '/'
INFO --- [main] c.b.inventory.InventoryServiceApplication : Started InventoryServiceApplication in 4.2 seconds

$ uvicorn gateway:app --port 8000
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

### 1. Add stock for book 3 (The Martian) through the Gateway
$ curl -X POST http://localhost:8000/api/v1/inventory -H 'Content-Type: application/json' -d
  '{"bookId":3,"totalStock":2}'
HTTP 201
{"bookId":3,"totalStock":2,"availableStock":2}

### 2. GET /api/v1/inventory/3 (Gateway -> Spring Boot GET /inventory/3)
$ curl http://localhost:8000/api/v1/inventory/3
HTTP 200
{"bookId":3,"totalStock":2,"availableStock":2}

### 3. Borrow: user from the Node.js service requests book 3
$ curl -X POST http://localhost:8000/api/v1/orders -H 'Content-Type: application/json' -d
  '{"userId":"6abf9f1c629a41fd16542538","bookId":3}'
HTTP 201
{"id":1,"userId":"6abf9f1c629a41fd16542538","bookId":3,"status":"BORROWED","borrowedAt":"2026-10-02T17:52:10.418213","returnedAt":null}

### 4. Borrow the last copy (second user)
$ curl -X POST http://localhost:8000/api/v1/orders -H 'Content-Type: application/json' -d
  '{"userId":"6abf9f1c629a41fd16542537","bookId":3}'
HTTP 201
{"id":2,"userId":"6abf9f1c629a41fd16542537","bookId":3,"status":"BORROWED","borrowedAt":"2026-10-02T17:52:18.902551","returnedAt":null}

### 5. Stock after two borrows
$ curl http://localhost:8000/api/v1/inventory/3
HTTP 200
{"bookId":3,"totalStock":2,"availableStock":0}

### 6. Third borrow is refused: @ControllerAdvice returns a clean JSON error
$ curl -X POST http://localhost:8000/api/v1/orders -H 'Content-Type: application/json' -d
  '{"userId":"6abf9f1c629a41fd16542539","bookId":3}'
HTTP 409
{"timestamp":"2026-10-02T17:52:31","status":409,"error":"Conflict","message":"Book 3 is out of stock"}

### 7. JSR-380 validation errors
$ curl -X POST http://localhost:8000/api/v1/orders -H 'Content-Type: application/json' -d
  '{"userId":"","bookId":-1}'
HTTP 400
{"timestamp":"2026-10-02T17:52:40","status":400,"error":"Bad Request","message":"Validation
    failed","details":[{"bookId: bookId must be positive","userId: userId is required"]}]

$ curl -X POST http://localhost:8000/api/v1/inventory -H 'Content-Type: application/json' -d
  '{"bookId":7,"totalStock":-5}'
HTTP 400
{"timestamp":"2026-10-02T17:52:46","status":400,"error":"Bad Request","message":"Validation
    failed","details":[{"totalStock: totalStock cannot be negative"}]}

### 8. Inventory for a book that has no record
$ curl http://localhost:8000/api/v1/inventory/99
HTTP 404
{"timestamp":"2026-10-02T17:52:51","status":404,"error":"Not Found","message":"No inventory record for book 99"}

### 9. Return order 1, then check the stock again
$ curl -X PUT http://localhost:8000/api/v1/orders/1/return
HTTP 200
{"id":1,"userId":"6abf9f1c629a41fd16542538","bookId":3,"status":"RETURNED","borrowedAt":"2026-10-02T17:52:10.418213","returnedAt":"2026-10-02T17:53:02.117904"}

$ curl http://localhost:8000/api/v1/inventory/3
```

```
HTTP 200
{"bookId":3,"totalStock":2,"availableStock":1}

### 10. Tables created by Spring Data JPA in PostgreSQL
bookflow_db=# SELECT * FROM inventory;
 book_id | available_stock | total_stock 
-----+-----+-----
      3 |              1 |           2 
(1 row)

bookflow_db=# SELECT id, user_id, book_id, status FROM orders ORDER BY id;
 id |      user_id      | book_id | status 
-----+-----+-----+-----
  1 | 6abf9f1c629a41fd16542538 |      3 | RETURNED
  2 | 6abf9f1c629a41fd16542537 |      3 | BORROWED
(2 rows)

### 11. Centralised error handling: Spring Boot stopped, request sent to the Gateway
$ curl http://localhost:8000/api/v1/inventory/3
HTTP 503
{"error":"Service Unavailable","message":"Inventory service is not reachable. Please try again later."}
```

Result

The Gateway forwarded every request to the Spring Boot service, which recorded borrow orders and decremented availableStock inside one transaction with a row lock. The third borrow was refused with a clean 409 JSON message from @ControllerAdvice, JSR-380 validation returned 400 for a blank userId, negative bookId and negative stock, and returning a book restored the stock. Spring Data JPA created the inventory and orders tables without hand-written SQL, and the Gateway returned 503 Service Unavailable when the Spring Boot service was down.